

# **BUILDING WITH AI**

**What the Work Actually Looks Like in Industry**

---

Webinar

# Sections

1. **Introduction**
2. **How AI Systems Actually Work: Common Misconceptions**
3. **Retrieval-Augmented Generation**
4. **Workflow-Based AI Systems**
5. **AI Agents**
6. **Evaluating AI Systems (if time permits...)**

# About the Speaker

*Where I started from -> 20 Years in Software Development*

- Education — B.E. Computer Science & M.E. Software Engineering
- Early Career — Trainee Engineer at TCS (low-level maintenance, manual testing, fixing build scripts)
- Experience — Senior → Lead → Architect across enterprise services & SaaS product engineering
- Current Role — Development Team Lead managing AI integrations, backend pipelines, and cloud services

LinkedIn:

<https://www.linkedin.com/in/mukul-varshney-59195512>

Github: <https://github.com/coolMukul>

Outsystems: <https://www.outsystems.com/profile/ogoe90tzrh>



1

# A Normal Working Day of a developer in AI era

- Read Jira ticket
- Plan (option A, B or C)
- Design if needed
- Implement + Unit tests
- Code Review
- Dev Testing
- Code Merge (Code review with more stringent checks)
- Send for Testing



# Scope of This Session

GenAI vs ML

## WHAT THIS TALK IS NOT

- **Internal Model Mechanics:** How a model works internally (math, backpropagation, attention mechanisms).
- **Model Training:** Fine-tuning weights, hyperparameter tuning, or training from scratch.
- **Prompt Engineering:** Cute prompt tricks, basic hacks, or magical phrasing.
- **Coding With AI:** Basic tutorials on using Copilot or ChatGPT to write code line-by-line.

## WHAT THIS TALK IS

- **System Architecture:** What GenAI integration requires at the application and backend level.
- **AI Engineering Practices:** Retrieval pipelines, dynamic workflows, tool orchestration, and testing.
- **Pragmatic Design Choices:** Practical choices for handling non-deterministic APIs in production systems.

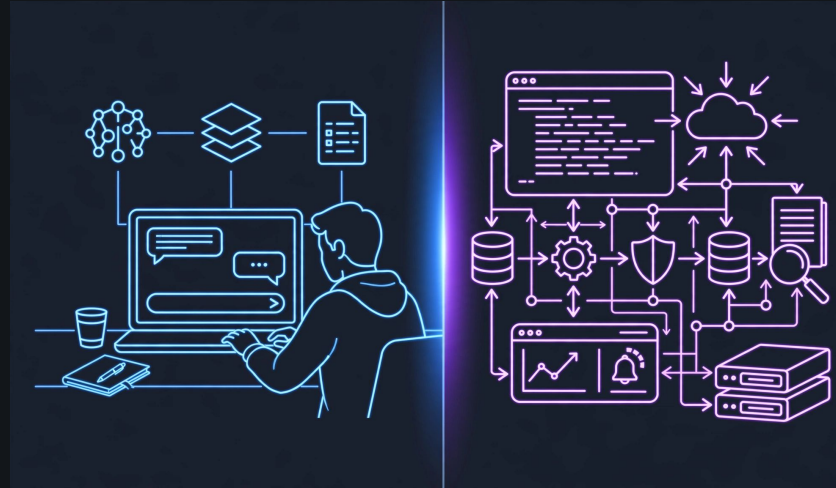


# 2

## Using AI Tools vs. Building AI Systems

*Using AI  $\neq$  Building with AI*

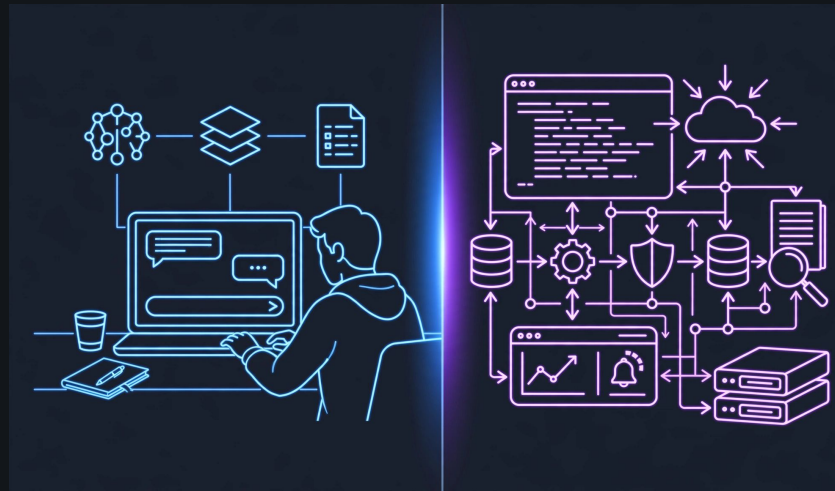
- **Using AI Tools** — ChatGPT, Copilot, web interfaces. You provide input; a managed app handles context, memory, and formatting.
- **Building AI Systems** — Writing backend code that talks to an unpredictable, non-deterministic HTTP API.



# Using AI Tools vs. Building AI Systems

*Using AI  $\neq$  Building with AI*

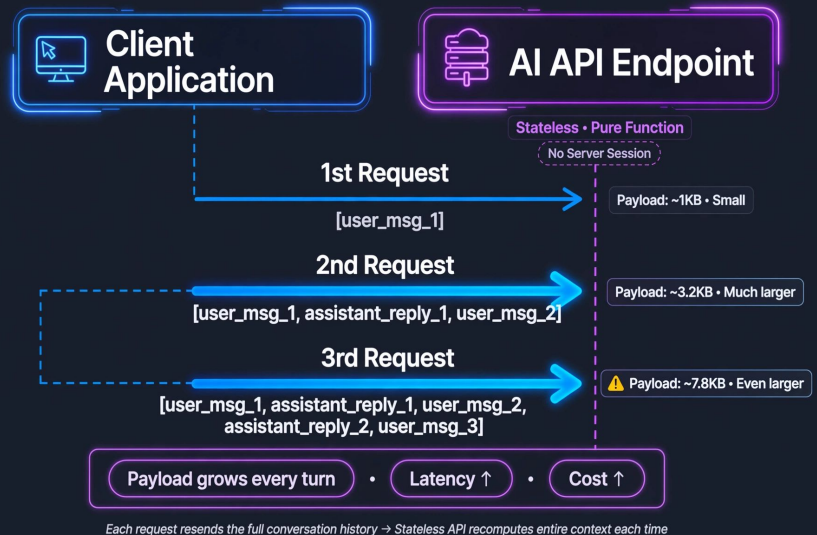
- **Using AI Tools** — ChatGPT, Copilot, web interfaces. You provide input; a managed app handles context, memory, and formatting.
- **Building AI Systems** — Writing backend code that talks to an unpredictable, non-deterministic HTTP API.
- **The Reality** — The model is just a remote third-party API endpoint. Everything else (data pipelines, state persistence, error boundaries) is your application code.



## Misconception 1 — That AI Remembers Your Conversation

*It doesn't remember anything. It is a stateless pure function.*

- No Server Session — The remote model endpoint holds zero state or stored session profiles between calls.
- Payload Inflation — Your application code must re-send the entire conversation array on every single request.
- Engineering Impact — As history grows, payload sizes grow, latency increases, and token API costs multiply per turn.



# 2

## Misconception 2 — That AI Stores and Retrieves Facts

*It doesn't look anything up. It predicts statistical text sequences.*

- No Internal Database — LLM has: No internal SQL table, key-value store, or document index inside weights.
- Next-Token Prediction — Generative models predict statistically probable next tokens based on training distribution patterns.
- Private Data Gap — The model will never automatically know your internal company documents or private customer data unless passed at runtime.

## Misconception 3 — That AI Can Perform Actions

*It cannot do anything on its own. It only emits text tokens.*

- Zero Execution Engine — Models cannot execute SQL statements, send emails, or run scripts directly.
- Text as Intent — The model merely outputs structured text (JSON / function signatures) representing intent.
- Your Code Does the Work — Your application parses text strings, validates permissions, and calls actual SDK functions.

## Misconception 4 — Prompting Is the Core Skill

*Prompting is ~5% of the codebase. Software engineering is ~95%.*



### PROMPTING ~5%

- string formatting & templates
- system role instructions
- adjusting phrasing for tone

← WATER LINE



### SOFTWARE ENGINEERING ~95%



Search & Retrieval  
DB indexing



Data Plumbing  
JSON serialization



API Integration  
Error handling & retries



Testing  
Eval pipelines & CI/CD

## Misconception 5 — Wrong Answers Mean a Faulty Model

*When the answer is wrong in production, it's usually NOT the model.*

Top Suspects in Production Failures:

1. Retrieval returned the wrong document (Most common)
2. The search index was stale or incomplete
3. Tool execution timed out or threw unhandled exception
4. Information was missing from source corpus
5. Model generation error (Least common)



# Reference Architecture of an Enterprise Application

*Out of 19 operational boxes, only 5 are AI-specific. Everything else is standard software engineering.*

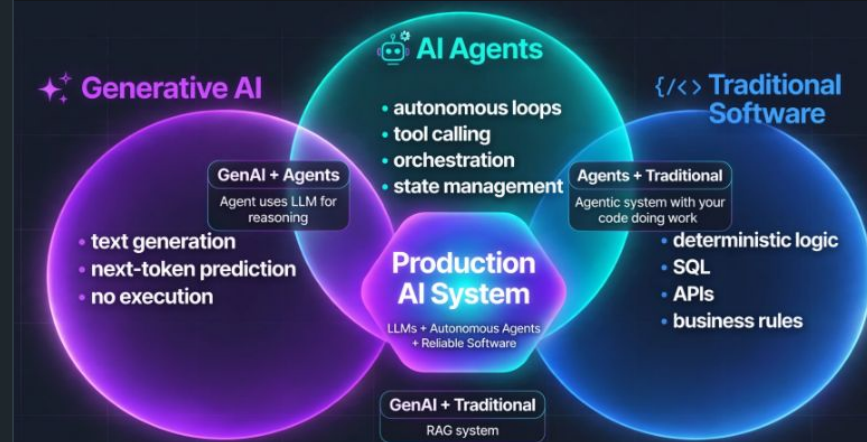
- 1. Edge/CDN/Gateway (Auth, Rate Limiting, CORS)
- 2. Client Layer (Web UI, Mobile, Desktop)
- 3. App Core/Backend (REST API, Auth, Business Logic)
  - └ [AI Node / Retrieval Module / Schema Parser]
- 4. Data Stores (PostgreSQL, Redis, Vector Index, S3)
- 5. Observability & Platform (Logging, CI/CD, Eval Suites)



# Key Terminology — GenAI, AI Agents & Agentic

*Cutting through current industry jargon*

- Generative AI — Capabilities classification: models designed to produce text, image, audio, or code tokens given a prompt.
- AI Agent — Application architecture combining a model with tools, memory, execution loop, and stopping condition.
- Agentic — Adjective describing system behavior along a spectrum of operational autonomy.



## The Problem — AI Systems and Private Data

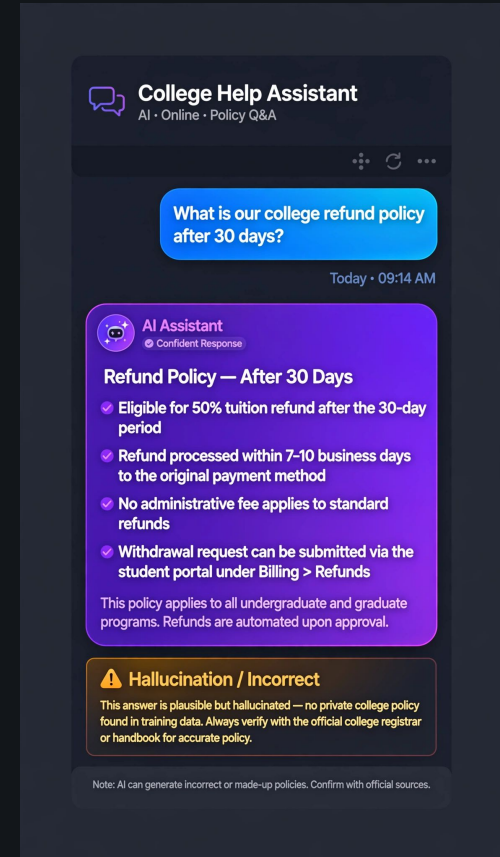
*Ask a standard model something it cannot possibly know.*

- Standard models are trained on public web snapshots and lack access to private corporate databases or internal policy files.
- When asked about private internal facts, models do not automatically fail or refuse to answer.

# The Problem — AI Systems and Private Data

*Ask a standard model something it cannot possibly know.*

- Standard models are trained on public web snapshots and lack access to private corporate databases or internal policy files.
- When asked about private internal facts, models do not automatically fail or refuse to answer.
- They generate plausible-sounding answers with total confidence.



## Why Retraining Is Not the Answer

*Facts change daily. Fine-tuning runs take time and money.*

- ❌ High Financial Cost — Fine-tuning or pre-training enterprise models requires massive GPU compute budgets.
- ❌ Slow Latency — Training jobs take hours, days, or weeks to process updated dataset exports.
- ❌ Stale Data — If internal policy updates at 9 AM, a model trained yesterday is obsolete.
- ❌ Knowledge vs. Behavior — Fine-tuning adapts style/tone/format, not factual databases.

# Why Retraining Is Not the Answer

*Facts change daily. Fine-tuning runs take time and money.*

- ❌ High Financial Cost — Fine-tuning or pre-training enterprise models requires massive GPU compute budgets.
- ❌ Slow Latency — Training jobs take hours, days, or weeks to process updated dataset exports.
- ❌ Stale Data — If internal policy updates at 9 AM, a model trained yesterday is obsolete.
- ❌ Knowledge vs. Behavior — Fine-tuning adapts style/tone/format, not factual databases.

## Fine-Tuning vs RAG

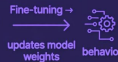
Style / Behavior vs Knowledge Retrieval



**Fine-Tuning =  
Style / Behavior**

Fine-tuning teaches HOW to say

- changes tone, personality, format
- adjusts weights, behavior
- learns style from examples



HOW to say — tone, voice, format, personality



**RAG = Knowledge  
Retrieval**

RAG teaches WHAT to say

- retrieves external documents
- injects actual facts at query time
- grounds response in real data



WHAT to say — factual knowledge, sources, up-to-date info

• Fine-tuning: changes the model itself | • RAG: adds external knowledge on the fly

# The Retrieval-Augmented Generation (RAG) Pipeline

*If the model only knows what's in the prompt, put the right data in the prompt.*

**01**

CHUNK

Split files into blocks.

**02**

INDEX

Store chunks in DB /  
Vector.

**03**

RETRIEVE

Fetch top-k relevant  
docs.

**04**

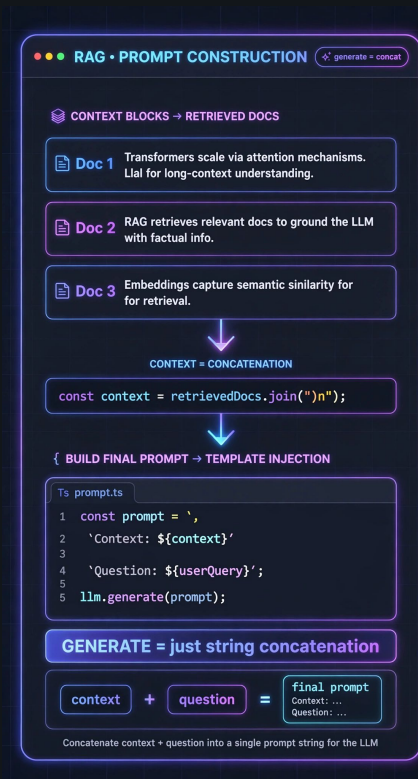
GENERATE

Pass context into LLM  
call.

# RAG is an Information Retrieval Problem You Already Know

*Steps 1–3 are standard Database and Search problems you already study.*

- Chunking & Indexing — Similar to designing database schemas, creating inverted indexes, or generating text token keys.
- Retrieval — Running ranked search queries using SQL full-text search, TF-IDF keyword matching, or vector cosine similarity.
- Generation — Basic string concatenation in application code:  
`finalPrompt = systemInstruction + retrievedDocs + userQuery;`



# Solving for failures

*When the answer is wrong in production, it's usually NOT the model.*

Top Suspects in Production Failures:

1. Retrieval returned the wrong document (Most common)
2. The search index was stale or incomplete
3. Tool execution timed out or threw unhandled exception
4. Information was missing from source corpus
5. Model generation error (Least common)



# Where RAG Fails in Practice — Retrieval Quality

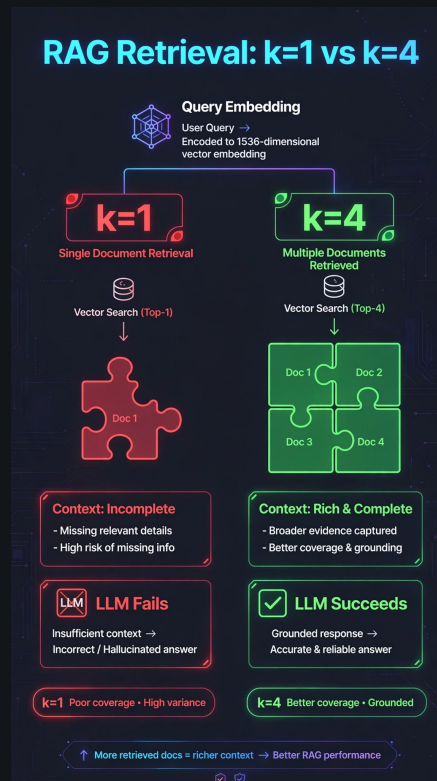
*Most 'AI generation failures' are actually search retrieval failures.*

## SCENARIO A (K = 1)

- Search fetches 1 document chunk
- Misses updated amendment clause
- Outcome: Confident, hallucinated or partial answer

## SCENARIO B (K = 4)

- Search fetches top 4 chunks
- Includes main policy + context
- Outcome: Accurate, fully grounded response



# Prompt vs Context



## Prompt

instructions • task • system message • HOW to behave

- **Instructions:**  
You are a helpful assistant.  
Reply concisely in bullet points.
- **Task:**  
Classify sentiment as POSITIVE /  
NEGATIVE / NEUTRAL.
- **System Message:**  
Always respond in JSON format.  
No extra commentary.

### Example:

- User: "I love the update!" →  
{ "sentiment": "POSITIVE" }



## <> Final Input to LLM

```
finalPrompt = systemPrompt  
+ Context + retrievedDocs  
}
```



## Context

data • retrieved docs • facts • WHAT to use

- **Data:**  
- User profile: tier=FREE,  
region=EU, locale=en-GB
- **Retrieved Docs:**  
- Doc #12: "Refund Policy v3.1"  
(updated Jan 2024)
- **Facts:**  
- Latency SLA: p99 < 200ms  
• Max tokens: 4096

### Example:

- Policy states: Refunds available  
within 30 days of purchase

• Prompt = Behavior Control

• Context = Knowledge Grounding

# Enterprise Practices

## Prompt Engineering

- **System Directive Isolation:** Keep system instructions (persona, constraints, formatting) separate from user input to prevent prompt injection and ensure consistent output schemas (e.g., forcing structured JSON).
- **Few-Shot Prompting:** Include 1–3 concrete input/output examples within the prompt payload to drastically improve schema adherence and reduce hallucination.
- **Guardrails & Safety Layers:** Enforce strict policy guardrails (e.g., "Do not guess missing data", "Refuse off-topic requests") at the prompt layer before sending requests to the LLM.

## Dynamic Context Assembly & RAG

- **RAG:** Instead of fine-tuning models on volatile data, dynamically fetch real-time documents (e.g., vector search over policy databases) and pass them as context.
- **Context Pruning & Re-ranking:** Re-rank vector search results before injecting them into the context window to maximize relevance and avoid hitting token limits.

# 3

## Enterprise Practices

### Orchestration & Production Engineering (The Center)

- **Context Window & Cost Optimization:** Context tokens are expensive and increase latency. Industry systems compress context and enforce maximum token budgets (e.g., capping docs at 2,000 tokens).
- **Latency Management (p99 SLA):** Pre-fetch static prompts and user metadata asynchronously while parallelizing vector retrieval to meet enterprise SLAs (e.g., < 200ms total pipeline overhead).
- **Auditability & Traceability:** Store the merged final Prompt payload in logging telemetry for security auditing, hallucination debugging, and compliance evaluation.

# 3

## Enterprise Practices

### Key Takeaway for higher accuracy

- **Chunking strategy:** Parse by semantic context, not character limits
- **Retrieval quality**
- **Re-ranking:** Combine Hybrid Search + Cross-Encoder Re-ranking

# 3 Enterprise Practices

## Production LLM Guardrail Layers

What should every production AI system implement?

### Input Guardrails

Always

#### Injection Detection

Blocks prompt injection and jailbreak attempts

Always

#### PII Redaction

Strips sensitive data before it enters LLM context

Always

#### Policy Gate

Length caps, rate limits, scope enforcement

Public-facing

#### Content Safety

Screens for hate, violence, self-harm, CSAM

### Output Guardrails

Always

#### PII Sweep

Second pass on generated response

Public-facing

#### Output Safety

Re-screens output for harmful content

RAG systems

#### Groundedness Check

Verifies claims match retrieved evidence

Feeds systems

#### Schema Validation

Ensures valid JSON, fields, and tool outputs

**Key takeaway: Run guardrails on both input and output.**

Source: OWASP LLM Top 10 - 2025

# 4

## Designing Fixed Pipelines — Flowcharts in Code

*You have been drawing these since first year.*

- **Structured Execution** — Chaining deterministic programming logic (if/else, switch statements) with targeted LLM calls.
- **Controlled Boundaries** — The LLM is restricted to solving specific single tasks (classifying text intent or parsing structured fields).
- **Predictable Architecture** — The overall graph topology is fully defined before runtime.

# 4

## Case Study — Automated Request Handling

*A fixed execution graph featuring human-in-the-loop fallback pathways.*

A mid-sized SaaS company receives hundreds of customer support emails daily (password resets, billing queries, feature requests, complaints). Manual handling is slow, inconsistent, and costly. The goal is to automate the majority of routine requests while safely escalating complex or high-risk cases to human agents.

# Case Study — Automated Request Handling

*A fixed execution graph featuring human-in-the-loop fallback pathways.*

**01**

## INGEST

Receive email via  
webhook.

Incoming support emails are captured in real time through a secure webhook endpoint and normalized into a structured request object.

**02**

## CLASSIFY

Categorize intent via  
LLM.

An LLM classifies the request into predefined intents (e.g., Billing, Account Access, Feature Request, Complaint) and extracts key entities (account ID, urgency, sentiment).

**03**

## LOOKUP

Query DB for account  
state.

The system retrieves the customer's current account status, subscription plan, recent tickets, and risk flags from the internal database.

**04**

## DECIDE

Draft AI response or  
escalate.

Based on classification + account data, the agent either (a) generates a personalized draft response OR (b) routes the case to a human agent with full context when confidence is low, risk is high, or policy requires escalation.

# Structured Output — Machine-Readable AI

*Downstream code cannot parse unstructured conversational text paragraphs.*

- Integration Requirement — Backend functions require strict data types (integers, booleans, enums).
- Schema Enforcement — Force model API outputs into strict JSON schemas using schema definition flags.
- Validation Layer — Application code parses incoming JSON. If parsing fails, code triggers automatic retry loop.



## Unstructured Text

Hey, just a quick note — the user order came through okay I think, it's for Sarah, she wanted the premium plan, not sure if it's active yet or pending, maybe check her email is sarah.example@mail.com? Also the amount was like roughly \$120... could be USD? let me know when it's done, thanks!



### Issues:

- Ambiguous fields
- No type safety
- Parsing errors
- Unreliable for automation



Risk: Downstream code fails validation



## Structured JSON Object

```
{ "customer": "Sarah",  
  "plan": "premium",  
  "status": "pending",  
  "email": "sarah.example@mail.com",  
  "amount_usd": 120.00,  
  "validated": true }
```



### Benefits:

- Strict types defined
- Schema validated
- Reliable parsing
- Ready for automation



Success: Downstream code processes reliably



**Backend Integration Requirement:** downstream code requires strict types • JSON schema enforcement ensures consistent data for services



# Advantages & Limitations of Fixed Workflows

*Weighing predictable control against execution adaptability.*



## Stability

### ADVANTAGES

- stability
- testable
- predictable costs
- debugging
- deterministic SLAs

## Flexibility

### LIMITATIONS

- inflexible
- cannot handle un-drawn paths
- complex branching
- manual fallback

# When Steps Cannot Be Defined in Advance

*Real-world problem solving often requires dynamic multi-step iteration.*

- The Problem — Many real requests cannot be mapped to static, predefined flowcharts. The required sequence of actions only becomes clear after intermediate results are obtained.

- Scenario — A student is struggling in Algorithms. Investigate possible causes, check academic standing and attendance, look for available tutoring slots this week, and propose a recovery plan to both the student and the advisor.

## Why Fixed Workflows Break ?

The agent may need to:

- Pull grades → notice a sudden drop
- Check attendance records
- Query tutoring calendar
- Cross-reference with previous advising notes
- Decide whether to escalate or draft a personalized plan
- The exact path and number of steps emerge only during execution.

- Agent Behavior — Plan → Act → Observe → Re-plan (iterative loop) until the goal is satisfied or human intervention is required.

# The Agent Loop Architecture

*Demystifying agent execution into core programming building blocks.*

**01**

SCHEMA

Prompt + Tool Schemas  
to LLM.

**02**

INTENT

Model returns: 'Call Tool  
X'.

**03**

EXECUTE

Backend executes Tool X  
natively.

**04**

LOOP

Append output & repeat  
turn.

# Tools — Connecting AI Systems to Real Actions

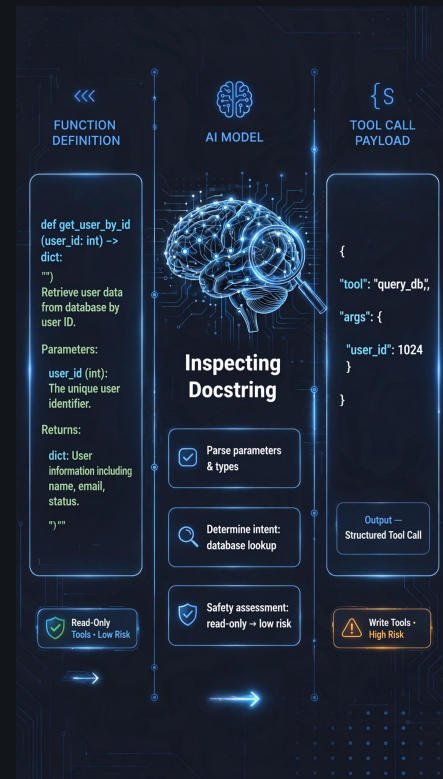
*Standard code functions wrapped with structural metadata and docstrings.*

## READ-ONLY TOOLS (LOW RISK)

- Query DB record by key
- Vector index lookup
- Fetch weather via REST API

## WRITE TOOLS (HIGH RISK)

- Execute SQL UPDATE/DELETE
- Trigger payment transaction
- Send outbound email



# Demonstration — An Agent Selecting Its Own Tools

*Dynamic tool selection from an active registry without hardcoded conditional logic.*

- Available Tools — [get\_student\_record, get\_course\_grades, check\_attendance, search\_tutoring\_slots, get\_advisor\_notes, draft\_recovery\_plan]
- User Query — 'A student is struggling in Algorithms. Investigate possible causes, check academic standing and attendance, look for available tutoring this week, and propose a recovery plan for both the student and the advisor.'
- Agent Reasoning & Tool Selection
  - Step 1 → Calls get\_student\_record + get\_course\_grades
  - Step 2 → Notices sharp grade drop → Calls check\_attendance
  - Step 3 → High absences found → Calls search\_tutoring\_slots + get\_advisor\_notes
  - Step 4 → Synthesizes findings → Calls draft\_recovery\_plan
  - Returns consolidated response with evidence and recommended actions


**TOOL REGISTRY**

|                                                                          |                                                                        |                                                                           |
|--------------------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <b>get_student_record</b><br>• fetch student profile, enrollment, status | <b>get_course_grades</b><br>• retrieve grades for all enrolled courses | <b>check_attendance</b><br>• pull attendance records and absences         |
| <b>search_tutoring_slots</b><br>• query available tutoring session slots | <b>get_advisor_notes</b><br>• retrieve advisor notes and observations  | <b>draft_recovery_plan</b><br>• generate intervention/recovery plan draft |


**USER QUERY**
[2024-11-21 09:13:58] [INFO] incoming\_query

> user: I am struggling in Algorithms. My grades have been dropping and I am not sure what to do.


**AGENT TRACE**
[2024-11-21 09:14:02] [INFO] step\_start

**[STEP 01] • INITIAL DATA FETCH**

> agent\_thinking: User reports struggling in Algorithms. First step is to pull the student record to identify the student, then fetch course grades to check for grade trend.

→ CALL: get\_student\_record(student\_id="12847")

[2024-11-21 09:14:03] [SUCCESS] tool\_response • get\_student\_record  
 { "student\_id": "12847", "name": "Alex Chen", "major": "Computer Science", "year": "Year 3", "status": "enrolled", "overall\_gpa": "3.1" }

→ CALL: get\_course\_grades(course="Algorithms", student\_id="12847")

[2024-11-21 09:14:05] [SUCCESS] tool\_response • get\_course\_grades  
 { "course": "Algorithms", "assignments": { "HW1": "78%", "HW2": "65%", "Midterm": "52%", }, "trend": "declining ~26% from HW1 to Midterm", "status": "concern: significant grade drop" }

→ CALL: get\_course\_grades(course="Algorithms", student\_id="12847")

[2024-11-21 09:14:09] [SUCCESS] tool\_response • check\_attendance  
 { "total\_classes": 12, "attended": 5, "absent": 7, "attendance\_rate": "42%", "pattern": "high absences from Week 4-6", "flag": "low attendance", "location": "Library Room 204", }


**[STEP 03] • SUDPORT RESOURCE GATHERING**
[2024-11-21 09:14:13] [INFO] step\_start

> agent\_thinking: Low attendance is likely contributing. To support the student, I will search for available tutoring slots and retrieve advisor notes for guidance tutoring and regular check-ins to improve engagement.

→ CALL: search\_tutoring\_slots(student\_id="Algorithms", timeframe="this\_week")


**[STEP 04] • SYNTHESIS & RECOVERY PLAN**
[2024-11-21 09:14:17] [INFO] step\_start

> agent\_thinking: Synthesizing all data — grade decline + 42% attendance + knowledge gaps on recursion. Now I will draft a recovery plan to address attendance, tutoring, and practice focus.

→ CALL: draft\_recovery\_plan(student\_id="12847", course="Algorithms")

[2024-11-21 09:14:18] [SUCCESS] tool\_response • draft\_recovery\_plan

```

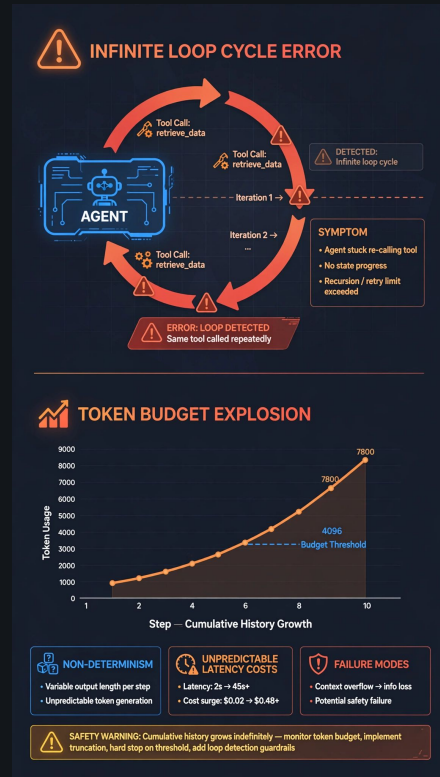
1. plan_name: "Algorithms Recovery Plan", actions: { "T1": "00-16:00", tutoring: "Tue & Thu 2:00-3:00pm, TA Lili, 24101, 24102, 24103, 24104 before each class",
2. improve_attendance_checkpoint: require check-in before each class,
3. practice regimen: 5 recursion DP problems daily using TA-provided worksheet,
4. progress_review: scheduled for 2024-11-18,
→ expected_outcome: "improve engagement, raise grade to 70% by next assessment",
status: "plan_ready"
  
```


[2024-11-21 09:14:21] [INFO] trace\_complete • status: SUCCESS • 4 steps • 6 tool calls executed successfully

# Engineering Challenges — Production Realities

*The hard realities of deploying autonomous agent loops into client production environments.*

- Non-Determinism — Same user query might take 2 execution turns today and 6 turns tomorrow.
- Unpredictable Latency & Costs — Multi-step agent loops send cumulative conversation history on every turn.
- Failure Modes & Infinite Loops — Agents can get stuck repeatedly executing same tool with identical inputs.
- Safety & Security — Malicious inputs injected into retrieved documents can trick loop into calling admin tools.



# Measuring Quality in AI Systems

*How do you know your system updates actually improved performance?*

- The Trap — Testing system performance manually by trying 3 favorite test queries in a terminal prompt is NOT evaluation.
- The Engineering Standard — Quantitative evaluation requires running automated benchmark test suites against ground-truth validation datasets.
- Core Software Engineering Principle — 'If you do not measure it systematically, you cannot improve it safely.'



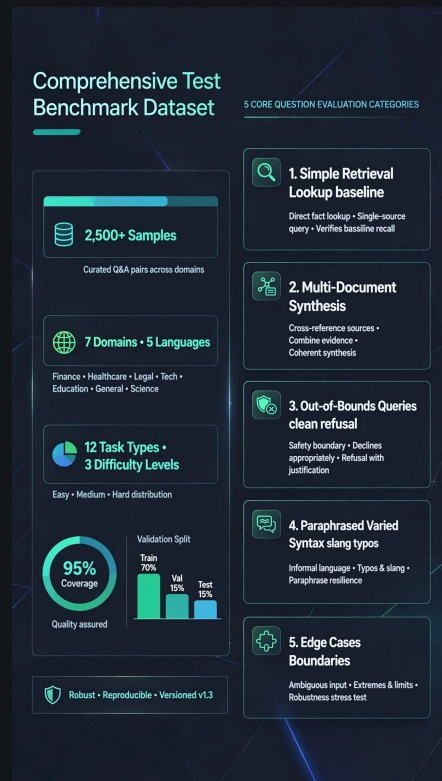
**If time permits —>**

Reference Material for Self-Study & Project Construction

# Building an Evaluation Set

*5 core question categories required for a comprehensive test benchmark dataset.*

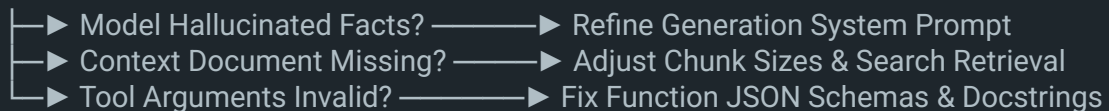
1. Simple Retrieval Lookup — Baseline single-document factual queries.
2. Multi-Document Synthesis — Complex queries requiring joining info across multiple chunks.
3. Out-of-Bounds Queries — Context is missing entirely; correct passing response is clean refusal.
4. Paraphrased / Varied Syntax — Identical underlying intent phrased with slang or typos.
5. Edge Cases & Boundaries — Multi-step constraints, dates, and threshold evaluations.



# Diagnosing Failures with Evidence

*Using evaluation suites to pinpoint exact failing components in your pipeline.*

- Run Evaluation Suite → Identify Failing Tests:



- Result — Fix root causes using evidence rather than guessing.

# Skills That Matter vs. Skills That Matter Less

*Focusing study time and project focus effectively during remaining academic semesters.*

## SKILLS THAT MATTER

- Python, C#, TypeScript
- HTTP, REST, JSON, Webhooks
- SQL, indexing, DB design
- Git, Docker, CI/CD
- Reading stack traces & logs

## SKILLS THAT MATTER LESS

- Memorizing transformer math
- Chasing weekly model releases
- Prompt Engineering certificates

# APPENDIX

Reference Material for Self-Study & Project Construction

# A1 — Understanding Token Usage and Cost

*Appendix: Reference Material*

- Token Conversion — 1 Token  $\approx$  0.75 words (or  $\sim$ 4 characters of standard English text).
- Cost Structure — APIs charge separately for Input Tokens (Prompts/Context) and Output Tokens (Generated text).
- Price Difference — Output tokens require sequential auto-regressive model forward passes and cost 3x to 4x more than input tokens.

# A2 — Cost as an Architectural Decision

*Appendix: Reference Material*

- Token Multipliers — Re-sending conversation history, injecting large document chunks, and multi-turn agent loops multiply token consumption exponentially.
- Architectural Cost Controls — Reduce chunk retrieve count ( $k$ ), prune old chat history, route simple sub-tasks to smaller models, cap max token limits.

# A3 — Getting Started with Free API Access

*Appendix: Reference Material*

- Free Developer Tiers — Use platforms like Google AI Studio or Groq Cloud Console for free access keys without credit cards.
- Managing Rate Limits — Watch out for Requests Per Minute (RPM) and Tokens Per Minute (TPM) quota limits on free tiers.
- Resilient Code — Implement exponential backoff handling in code to handle HTTP 429 rate limit responses gracefully.

# A4 — Glossary of Key Terms

*Appendix: Reference Material*

- Context Window — Absolute maximum token payload limit a model can process in single request turn.
- Grounding — Degree to which an output response is strictly derived from provided reference documents.
- Hallucination — Confident generation of unsubstantiated, ungrounded, or incorrect information.
- Vector Embedding — High-dimensional numerical vector representing semantic meaning of text.

# A6 — Further Reading & Technical Resources

*Appendix: Reference Material*

- Architecture Guides — Anthropic Engineering Blog ('Building Effective Agents').
- Developer Handbooks — OpenAI Cookbook (Structured outputs and tool use guides).
- Framework Docs — LangChain & LlamaIndex documentation (Study architectural design patterns; avoid over-relying on heavy abstraction libraries early on).